

Homework 2 - Cesar Cabrera Garcia

COP 4045-042 Python Programming | Fall 2026

Problem 1: Source File Parsing

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Number source lines and extract Python function definitions

import ast
import io
import os
import tokenize

student_name = "Cesar Cabrera Garcia"
source_filename = os.path.abspath(__file__)
numbered_filename = source_filename + ".txt"

def line_number(input_filename: str, output_filename: str) -> None:
    """Write a numbered copy of a text file; report and re-raise file errors."""
    line_counter = 1
    source_lines = []

    try:
        # Check aliases too, because opening the source for writing destroys it.
        if os.path.realpath(input_filename) == os.path.realpath(output_filename):
            raise ValueError("The input and output files must be different.")
        if os.path.exists(output_filename):
            if os.path.samefile(input_filename, output_filename):
                raise ValueError("The output points to the input file.")
        with open(input_filename, "r", encoding="utf-8") as source_file:
            source_lines = source_file.readlines()
        with open(output_filename, "w", encoding="utf-8") as output_file:
            for current_line in source_lines:
                output_file.write(str(line_counter) + ". " + current_line)
                line_counter = line_counter + 1
    except (OSError, UnicodeError, ValueError) as error:
        print("[ERROR] Could not create the numbered file:", error)
        raise

def parse_functions(filename: str) -> tuple:
    """Return (line, name, arguments, code) records sorted by function name.

    Include nested functions. Preserve docstrings and literal string contents,
    but remove comments and blank lines outside multiline string literals.
    File and syntax errors are reported and re-raised.
    """
    source_text = ""
    function_records = []
    sorted_records = []

    try:
        with open(filename, "r", encoding="utf-8") as source_file:
            source_text = source_file.read()
        syntax_tree = ast.parse(source_text, filename=filename)
        for node in ast.walk(syntax_tree):
            if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                function_text = ast.get_source_segment(source_text, node)
                function_lines = function_text.splitlines(keepends=True)
                protected_lines = []
                tokens = list(tokenize.generate_tokens(io.StringIO(function_text).readline))
                argument_start = None
                argument_end = None
                parenthesis_depth = 0

                # Tokens distinguish real comments from a '#' inside a string.
                for token in tokens:
```

```

        if token.type == tokenize.COMMENT:
            row, column = token.start
            function_lines[row - 1] = function_lines[row - 1][:column] + "\n"
        if token.type == tokenize.STRING:
            for row in range(token.start[0], token.end[0] + 1):
                protected_lines.append(row)
        if token.type == tokenize.OP and argument_end is None:
            if token.string == "(":
                if argument_start is None:
                    argument_start = token.end
                    parenthesis_depth = parenthesis_depth + 1
            elif token.string == ")":
                parenthesis_depth = parenthesis_depth - 1
                if parenthesis_depth == 0 and argument_start is not None:
                    argument_end = token.start

    original_lines = function_text.splitlines(keepends=True)
    start_row, start_column = argument_start
    end_row, end_column = argument_end
    if start_row == end_row:
        arguments = original_lines[start_row - 1][start_column:end_column]
    else:
        arguments = original_lines[start_row - 1][start_column:]
        for row in range(start_row, end_row - 1):
            arguments = arguments + original_lines[row]
        arguments = arguments + original_lines[end_row - 1][:end_column]

    cleaned_code = ""
    for index in range(len(function_lines)):
        current_line = function_lines[index]
        if current_line.strip() != "" or index + 1 in protected_lines:
            cleaned_code = cleaned_code + current_line
    if not cleaned_code.endswith("\n"):
        cleaned_code = cleaned_code + "\n"
    function_records.append((node.name, node.lineno, arguments, cleaned_code))

function_records.sort()
for record in function_records:
    sorted_records.append((record[1], record[0], record[2], record[3]))
return tuple(sorted_records)
except (OSError, UnicodeError, SyntaxError, tokenize.TokenError) as error:
    print("[ERROR] Could not parse the Python file:", error)
    raise

def main() -> None:
    """Test both operations on this script without overwriting the source."""
    print(student_name)
    print("[STATUS] Creating numbered source...")
    line_number(source_filename, numbered_filename)
    print("Numbered file:", numbered_filename)
    with open(numbered_filename, "r", encoding="utf-8") as numbered_file:
        for line_index in range(8):
            print(numbered_file.readline(), end="")
    print("[STATUS] Parsing functions...")
    print(parse_functions(source_filename))

if __name__ == "__main__":
    main()

```

Captured Program Output

```

Cesar Cabrera Garcia
[STATUS] Creating numbered source...
Numbered file: E:\Misc School Projects\COP 4045-042 Python Programming\Homework2\p1_CabreraGarcia_Cesar.py.txt
1. # Author Cesar Cabrera Garcia
2. # Course COP 4045-042 Python Programming
3. # Term Fall 2026
4. # Description Number source lines and extract Python function definitions
5.
6. import ast

```

```

7. import io
8. import os
[STATUS] Parsing functions...
((16, 'line_number', 'input_filename: str, output_filename: str', 'def line_number(input_filename: str, output_file
name: str) -> None:\n      """Write a numbered copy of a text file; report and re-raise file errors."""\n      line_c
ounter = 1\n      source_lines = []\n      try:\n          if os.path.realpath(input_filename) == os.path.realpath(outpu
t_filename):\n              raise ValueError("The input and output files must be different.")\n          if os.path.ex
ists(output_filename):\n              if os.path.samefile(input_filename, output_filename):\n                  raise V
alueError("The output points to the input file.")\n              with open(input_filename, "r", encoding="utf-8") as sou
rce_file:\n                  source_lines = source_file.readlines()\n                  with open(output_filename, "w", encoding="u
tf-8") as output_file:\n                      for current_line in source_lines:\n                          output_file.write(str(line_
counter) + ". " + current_line)\n                          line_counter = line_counter + 1\n                      except (OSError, UnicodeErro
r, ValueError) as error:\n                          print("[ERROR] Could not create the numbered file:", error)\n                          raise\n'),
(111, 'main', '', 'def main() -> None:\n      """Test both operations on this script without overwriting the source.
"""\n      print(student_name)\n      print("[STATUS] Creating numbered source...")\n      line_number(source_filename,
numbered_filename)\n      print("Numbered file:", numbered_filename)\n      with open(numbered_filename, "r", encoding
="utf-8") as numbered_file:\n          for line_index in range(8):\n              print(numbered_file.readline(), end=
"")\n          print("[STATUS] Parsing functions...")\n          print(parse_functions(source_filename))\n'), (39, 'parse_func
tions', 'filename: str', 'def parse_functions(filename: str) -> tuple:\n      """Return (line, name, arguments, code
) records sorted by function name.\n      Include nested functions. Preserve docstrings and literal string content
s,\n      but remove comments and blank lines outside multiline string literals.\n      File and syntax errors are rep
orted and re-raised.\n      """\n      source_text = ""\n      function_records = []\n      sorted_records = []\n      try:\n
          with open(filename, "r", encoding="utf-8") as source_file:\n              source_text = source_file.read()\n
          syntax_tree = ast.parse(source_text, filename=filename)\n              for node in ast.walk(syntax_tree):\n
                  if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):\n                      function_text = ast.get_sou
rce_segment(source_text, node)\n                      function_lines = function_text.splitlines(keepends=True)\n
                      protected_lines = []\n                      tokens = list(tokenize.generate_tokens(io.StringIO(function_text).re
adline))\n                      argument_start = None\n                      argument_end = None\n                      parenthesis_
depth = 0\n                      for token in tokens:\n                          if token.type == tokenize.COMMENT:\n
                              row, column = token.start\n                              function_lines[row - 1] = function_lines[row - 1
]][:column] + "\n\n"
                              if token.type == tokenize.STRING:\n                                  for row in rang
e(token.start[0], token.end[0] + 1):\n                                      protected_lines.append(row)\n
                              if token.type == tokenize.OP and argument_end is None:\n                                  if token.string == "(":
                                      argument_start = token.end\n                                  elif token.string == ")":
                                      argument_end = token.start\n
                                      if argument_start is None:\n                                          parenthesis_depth = parenthesis_depth + 1\n
                                          elif token.string == ")":
                                              parenthesis_depth = parenthesis_depth - 1\n
                                              if parenthesis_
s_depth == 0 and argument_start is not None:\n                                                  argument_end = token.start\n
                                                  original_lines = function_text.splitlines(keepends=True)\n                                                  start_row, start_column = argume
nt_start\n                                                  end_row, end_column = argument_end\n                                                  if start_row == end_row:\n
                                                      arguments = original_lines[start_row - 1][start_column:end_column]\n
                                                      else:\n
                                                          arguments = original_lines[start_row - 1][start_column:]
                                                          for row in range(start_row, e
nd_row - 1):\n                                                              arguments = arguments + original_lines[row]\n
                                                              arguments =
arguments + original_lines[end_row - 1][:end_column]\n
                                                              cleaned_code = ""\n
                                                              for inde
x in range(len(function_lines)):\n                                                                  current_line = function_lines[index]\n
                                                                  if
current_line.strip() != "" or index + 1 in protected_lines:\n                                                                      cleaned_code = cleaned_code
+ current_line\n
                                                                      if not cleaned_code.endswith("\n"):\n                                                                          cleaned_code = cleaned_c
ode + "\n\n"
                                                                      function_records.append((node.name, node.lineno, arguments, cleaned_code))\n
                                                                      f
unction_records.sort()\n
                                                                      for record in function_records:\n
                                                                          sorted_records.append((record[1], rec
ord[0], record[2], record[3]))\n
                                                                          return tuple(sorted_records)\n
                                                                          except (OSError, UnicodeError, SyntaxErr
or, tokenize.TokenError) as error:\n
                                                                              print("[ERROR] Could not parse the Python file:", error)\n
                                                                              rais
e\n'))

```

```

PS E:\MISC School Projects\4045-042 Python Programming\Homework2> C:\Users\cesar\AppData\Local\Python\pythoncore\3-14-64\python.exe "E:\MISC School Projects\4045-042 Python Programming\Homework2\p1_CabreraGarcia_Cesar.py"
Cesar Cabrera Garcia
[STATUS] Creating numbered source...
Numbered file: E:\MISC School Projects\4045-042 Python Programming\Homework2\p1_CabreraGarcia_Cesar.py.txt
1. # Author Cesar Cabrera Garcia
2. # Course COP 4045-042 Python Programming
3. # Term Fall 2025
4. # Description Number source lines and extract Python function definitions
5.
6. import ast
7. import io
8. import os
[STATUS] Parsing functions...
((16, 'line_number', 'input_filename: str, output_filename: str', 'def line_number(input_filename: str, output_file
name: str) -> None:\n      """Write a numbered copy of a text file; report and re-raise file errors."""\n      line_counter = 1\n      source_lines = []\n      try:\n          if os.path.realpath(input_filename) == os.path.realpath(output_filename):\n              raise ValueError("The input and output files must be different.")\n          if os.path.exists(output_filename):\n              if os.path.samefile(input_filename, output_filename):\n                  raise ValueError("The output points to the input file.")\n              with open(input_filename, "r", encoding="utf-8") as source_file:\n                  source_lines = source_file.readlines()\n                  with open(output_filename, "w", encoding="utf-8") as output_file:\n                      for current_line in source_lines:\n                          output_file.write(str(line_counter) + ". " + current_line)\n                          line_counter = line_counter + 1\n                      except (OSError, UnicodeError, ValueError) as error:\n                          print("[ERROR] Could not create the numbered file:", error)\n                          raise\n'), (111, 'main', '', 'def main() -> None:\n      """Test both operations on this script without overwriting the source.
"""\n      print(student_name)\n      print("[STATUS] Creating numbered source...")\n      line_number(source_filename,
numbered_filename)\n      print("Numbered file:", numbered_filename)\n      with open(numbered_filename, "r", encodi
ng="utf-8") as numbered_file:\n          for line_index in range(8):\n              print(numbered_file.readline(), end="")\n          print("[STATUS] Parsing functions...")\n          print(parse_functions(source_filename))\n'), (39, 'parse_functions', 'filename: str', 'def parse_functions(filename: str) -> tuple:\n      """Return (line, name, arguments, code) records sorted by function name.\n      Include nested functions. Preserve docstrings and literal string contents,\n      but remove comments and blank lines outside multiline string literals.\n      File and syntax errors are reported and re-raised.\n      """\n      source_text = ""\n      function_records = []\n      sorted_records = []\n      try:\n          with open(filename, "r", encoding="utf-8") as source_file:\n              source_text = source_file.read()\n              syntax_tree = ast.parse(source_text, filename=filename)\n              for node in ast.walk(syntax_tree):\n                  if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):\n                      function_text = ast.get_source_segment(source_text, node)\n                      function_lines = function_text.splitlines(keepends=True)\n                      protected_lines = []\n                      tokens = list(tokenize.generate_tokens(io.StringIO(function_text).readline))\n                      argument_start = None\n                      argument_end = None\n                      parenthesis_depth = 0\n                      for token in tokens:\n                          if token.type == tokenize.COMMENT:\n                              row, column = token.start\n                              function_lines[row - 1] = function_lines[row - 1][:column] + "\n\n"
                              if token.type == tokenize.STRING:\n                                  for row in range(token.start[0], token.end[0] + 1):\n                                      protected_lines.append(row)\n
                              if token.type == tokenize.OP and argument_end is None:\n                                  if token.string == "(":
                                      argument_start = token.end\n                                  elif token.string == ")":
                                      argument_end = token.start\n
                                      if argument_start is None:\n                                          parenthesis_depth = parenthesis_depth + 1\n
                                          elif token.string == ")":
                                              parenthesis_depth = parenthesis_depth - 1\n
                                              if parenthesis_depth == 0 and argument_start is not None:\n                                                  argument_end = token.start\n
                                                  original_lines = function_text.splitlines(keepends=True)\n                                                  start_row, start_column = argument_start\n                                                  end_row, end_column = argument_end\n                                                  if start_row == end_row:\n
                                                      arguments = original_lines[start_row - 1][start_column:end_column]\n
                                                      else:\n
                                                          arguments = original_lines[start_row - 1][start_column:]
                                                          for row in range(start_row, end_row - 1):\n                                                              arguments = arguments + original_lines[row]\n
                                                              arguments = arguments + original_lines[end_row - 1][:end_column]\n
                                                              cleaned_code = ""\n
                                                              for index in range(len(function_lines)):\n                                                                  current_line = function_lines[index]\n
                                                                  if
current_line.strip() != "" or index + 1 in protected_lines:\n                                                                      cleaned_code = cleaned_code
+ current_line\n
                                                                      if not cleaned_code.endswith("\n"):\n                                                                          cleaned_code = cleaned_code + "\n\n"
                                                                      function_records.append((node.name, node.lineno, arguments, cleaned_code))\n
                                                                      function_records.sort()\n
                                                                      for record in function_records:\n
                                                                          sorted_records.append((record[1], record[0], record[2], record[3]))\n
                                                                          return tuple(sorted_records)\n
                                                                          except (OSError, UnicodeError, SyntaxError, tokenize.TokenError) as error:\n
                                                                              print("[ERROR] Could not parse the Python file:", error)\n
                                                                              raise\n'))
PS E:\MISC School Projects\4045-042 Python Programming\Homework2>

```

Problem 2: Comprehensions

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Solve six list and dictionary comprehension exercises

student_name = "Cesar Cabrera Garcia"
lower_bound = 1
upper_bound = 10
words = ["Cash", "Ubuntu", "Mint", "Zorin OS"]
names = ["Cash Ubuntu Mint", "Ubuntu Mint Cash"]
first_words = ["Cash", "Ubuntu", "Mint"]
second_words = ["CASH", "MINT", "Zorin OS"]
distinct_words = ["Cash", "Ubuntu", "Mint"]
text = "Cash Ubuntu Mint Zorin OS"

# Keep every ordering because the question asks for all distinct quadruples.
equal_squares = [
    (a, b, c, d)
    for a in range(lower_bound, upper_bound + 1)
    for b in range(lower_bound, upper_bound + 1)
    for c in range(lower_bound, upper_bound + 1)
    for d in range(lower_bound, upper_bound + 1)
    if a != b and a != c and a != d and b != c and b != d and c != d
    and a**2 + b**2 == c**2 + d**2
]
short_words = [(word.lower(), len(word)) for word in words if len(word) < 5]
abbreviated_names = [
    name.split()[0] + " " + name.split()[1][0] + ". " + name.split()[2]
    for name in names
]
# Sorted letters preserve repeated letters, which an anagram must also match.
anagram_pairs = [
    (word_one, word_two)
    for word_one in first_words
    for word_two in second_words
    if sorted(word_one.lower()) == sorted(word_two.lower())
]
word_lengths = {word: len(word) for word in distinct_words}
vowel_positions = {
    index: text[index] for index in range(len(text))
    if text[index].lower() in "aeiou"
}

def main() -> None:
    """Display the results of all six comprehensions."""
    print(student_name)
    print("[STATUS] Comprehension results")
    print("a)", equal_squares)
    print("b)", short_words)
    print("c)", abbreviated_names)
    print("d)", anagram_pairs)
    print("e)", word_lengths)
    print("f)", vowel_positions)

if __name__ == "__main__":
    main()
```

Captured Program Output

```
Cesar Cabrera Garcia
[STATUS] Comprehension results
a) [(1, 8, 4, 7), (1, 8, 7, 4), (2, 9, 6, 7), (2, 9, 7, 6), (4, 7, 1, 8), (4, 7, 8, 1), (6, 7, 2, 9), (6, 7, 9, 2),
(7, 4, 1, 8), (7, 4, 8, 1), (7, 6, 2, 9), (7, 6, 9, 2), (8, 1, 4, 7), (8, 1, 7, 4), (9, 2, 6, 7), (9, 2, 7, 6)]
b) [('cash', 4), ('mint', 4)]
c) ['Cash U. Mint', 'Ubuntu M. Cash']
d) [('Cash', 'CASH'), ('Mint', 'MINT')]
e) {'Cash': 4, 'Ubuntu': 6, 'Mint': 4}
```

f) {1: 'a', 5: 'U', 7: 'u', 10: 'u', 13: 'i', 18: 'o', 20: 'i', 23: 'O'}

```
PS E:\Misc School Projects\COP 4845-842 Python Programming\Homework2> & C:/Users/cedae/AppData/Local/Python/pythoncore-3.14-64/python.exe "e:/Misc School Projects/COP 4845-842 Python Programming/Homework2/p2_CabreraGarcia_Cesar.py"
Cesar Cabrera Garcia
[STATUS] Comprehension results
a) [(1, 8, 4, 7), (1, 8, 7, 4), (2, 9, 6, 7), (2, 9, 7, 6), (4, 7, 1, 8), (4, 7, 8, 1), (6, 7, 2, 9), (6, 7, 9, 2), (7, 4, 1, 8), (7, 4, 8, 1), (7, 6, 2, 9), (7, 6, 9, 2), (8, 1, 4, 7), (8, 1, 7, 4), (9, 2, 6, 7), (9, 2, 7, 6)]
b) [('cash', 4), ('mint', 4)]
c) ['Cash U. Mint', 'Ubuntu M. Cash']
d) [('Cash', 'CASH'), ('Mint', 'MINT')]
e) {'Cash': 4, 'Ubuntu': 6, 'Mint': 4}
f) {1: 'a', 5: 'U', 7: 'u', 10: 'u', 13: 'i', 18: 'o', 20: 'i', 23: 'O'}
PS E:\Misc School Projects\COP 4845-842 Python Programming\Homework2> █
```

Problem 3: Social Network

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Manage mutual friendships and save a social network as CSV

import csv
import os
import tempfile
from testif import testif

student_name = "Cesar Cabrera Garcia"
network_filename = os.path.join(os.path.dirname(__file__), "social_network.csv")

def add_user(sn: dict, username: str, fullname: str) -> bool:
    """Add a user with no friends; return False for an existing username."""
    try:
        if username in sn:
            return False
        sn[username] = (fullname, [])
        return True
    except (TypeError, ValueError) as error:
        print("[ERROR] Could not add the user:", error)
        raise

def add_friend(sn: dict, user1: str, user2: str) -> bool:
    """Add a mutual link; reject missing users and self-friendship."""
    try:
        if user1 not in sn or user2 not in sn or user1 == user2:
            return False
        if user2 not in sn[user1][1]:
            sn[user1][1].append(user2)
        if user1 not in sn[user2][1]:
            sn[user2][1].append(user1)
        return True
    except (TypeError, KeyError, IndexError, AttributeError) as error:
        print("[ERROR] Could not add the friendship:", error)
        raise

def get_friends(sn: dict, user1: str, distance: int) -> list:
    """Return unique friends within the given link distance, excluding user1."""
    found_friends = []
    visited_users = [user1]
    current_level = [user1]
    current_distance = 0

    try:
        if user1 not in sn or distance <= 0:
            return []
        # Search one level at a time so each person is reached by a shortest path.
        while current_distance < distance and current_level:
            next_level = []
            for current_user in current_level:
                for friend in sn[current_user][1]:
                    if friend not in visited_users:
                        visited_users.append(friend)
                        found_friends.append(friend)
                        next_level.append(friend)
            current_level = next_level
            current_distance = current_distance + 1
        return found_friends
    except (TypeError, KeyError, IndexError) as error:
        print("[ERROR] Could not look up friends:", error)
        raise

def save_network(filename: str, sn: dict) -> None:
```

```

"""Save headerless CSV rows: username, full name, then zero or more friends."""
try:
    with open(filename, "w", newline="", encoding="utf-8") as network_file:
        writer = csv.writer(network_file)
        for username in sn:
            writer.writerow([username, sn[username][0]] + sn[username][1])
except (OSError, UnicodeError, csv.Error, TypeError, IndexError) as error:
    print("[ERROR] Could not save the network:", error)
    raise

def load_network(filename: str) -> dict:
    """Read a network saved by save_network and reject broken friend links."""
    network = {}
    try:
        with open(filename, "r", newline="", encoding="utf-8") as network_file:
            reader = csv.reader(network_file, strict=True)
            for row in reader:
                if len(row) < 2 or row[0] in network:
                    raise ValueError("A network row is incomplete or duplicated.")
                network[row[0]] = (row[1], row[2:])
            for username in network:
                checked_friends = []
                for friend in network[username][1]:
                    if friend not in network or friend == username or friend in checked_friends:
                        raise ValueError("A friend link is missing, repeated, or self-referencing.")
                    if username not in network[friend][1]:
                        raise ValueError("Friend links must be mutual.")
                    checked_friends.append(friend)
            return network
    except (OSError, UnicodeError, csv.Error, ValueError) as error:
        print("[ERROR] Could not load the network:", error)
        raise

def test() -> bool:
    """Use the supplied testif helper to check all five network operations."""
    network = {}
    passed = True

    try:
        passed = testif(add_user(network, "Cash", "Cash"), "add user") and passed
        passed = testif(not add_user(network, "Cash", "Cash"), "duplicate user") and passed
        add_user(network, "Ubuntu", "Ubuntu")
        add_user(network, "Mint", "Mint")
        add_user(network, "Zorin OS", "Zorin OS")
        passed = testif(add_friend(network, "Cash", "Ubuntu"), "add friendship") and passed
        passed = testif("Cash" in network["Ubuntu"][1], "mutual friendship") and passed
        add_friend(network, "Ubuntu", "Mint")
        add_friend(network, "Mint", "Zorin OS")
        passed = testif(get_friends(network, "Cash", 2) == ["Ubuntu", "Mint"], "distance two") and passed
        add_friend(network, "Mint", "Cash")
        passed = testif(get_friends(network, "Cash", 10) == ["Ubuntu", "Mint", "Zorin OS"], "cycle handling") and
passed
        passed = testif(not add_friend(network, "Cash", "Cash"), "reject self link") and passed
        passed = testif(get_friends({}, "Cash", 1) == [], "unknown user") and passed
        passed = testif(not add_friend({}, "Cash", "Ubuntu"), "missing friends") and passed
        # Temporary storage keeps the extra-credit tests separate from the demo.
        with tempfile.TemporaryDirectory() as directory:
            filename = os.path.join(directory, "Cash.csv")
            save_network(filename, network)
            passed = testif(load_network(filename) == network, "CSV round trip") and passed
        return passed
    except (OSError, UnicodeError, csv.Error, ValueError, TypeError) as error:
        print("[ERROR] Could not complete network tests:", error)
        raise

def main() -> None:
    """Demonstrate each network operation and run the extra-credit tests."""
    network = {}
    try:
        print(student_name)

```

```

print("[STATUS] Creating social network...")
add_user(network, "Cash", "Cash")
add_user(network, "Ubuntu", "Ubuntu")
add_user(network, "Mint", "Mint")
add_friend(network, "Cash", "Ubuntu")
add_friend(network, "Ubuntu", "Mint")
print("Network:", network)
print("Cash friends at distance 1:", get_friends(network, "Cash", 1))
print("Cash friends at distance 2:", get_friends(network, "Cash", 2))
save_network(network_filename, network)
print("Reloaded network:", load_network(network_filename))
print("[STATUS] All extra-credit tests passed:", test())
except (OSError, UnicodeError, csv.Error, ValueError, TypeError) as error:
    print("[ERROR] The network demonstration failed:", error)
    raise

if __name__ == "__main__":
    main()

```

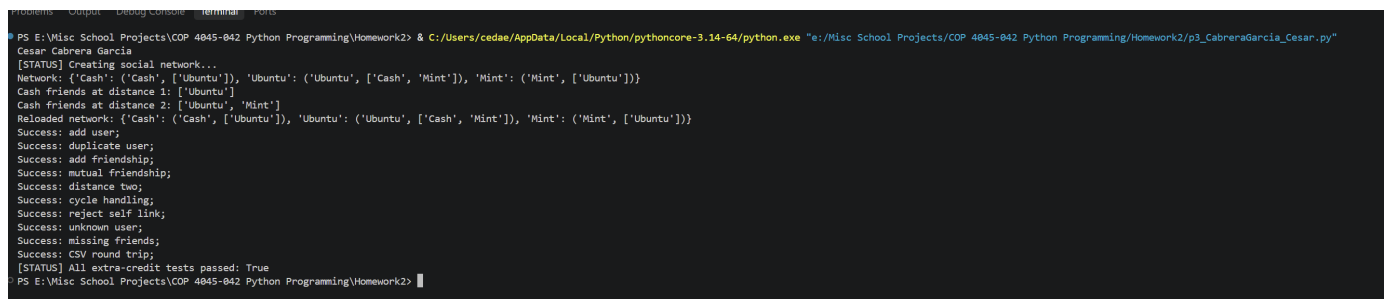
Captured Program Output

Cesar Cabrera Garcia

```

[STATUS] Creating social network...
Network: {'Cash': ('Cash', ['Ubuntu']), 'Ubuntu': ('Ubuntu', ['Cash', 'Mint']), 'Mint': ('Mint', ['Ubuntu'])}
Cash friends at distance 1: ['Ubuntu']
Cash friends at distance 2: ['Ubuntu', 'Mint']
Reloaded network: {'Cash': ('Cash', ['Ubuntu']), 'Ubuntu': ('Ubuntu', ['Cash', 'Mint']), 'Mint': ('Mint', ['Ubuntu'])}
Success: add user;
Success: duplicate user;
Success: add friendship;
Success: mutual friendship;
Success: distance two;
Success: cycle handling;
Success: reject self link;
Success: unknown user;
Success: missing friends;
Success: CSV round trip;
[STATUS] All extra-credit tests passed: True

```



The screenshot shows a terminal window with the following content:

```

PS E:\Misc School Projects\COP 4845-842 Python Programming\Homework2> & C:/Users/cedae/AppData/Local/Python/pythoncore-3.14-64/python.exe "e:/Misc School Projects/COP 4845-842 Python Programming/Homework2/p3_CabreraGarcia_Cesar.py"
Cesar Cabrera Garcia
[STATUS] Creating social network...
Network: {'Cash': ('Cash', ['Ubuntu']), 'Ubuntu': ('Ubuntu', ['Cash', 'Mint']), 'Mint': ('Mint', ['Ubuntu'])}
Cash friends at distance 1: ['Ubuntu']
Cash friends at distance 2: ['Ubuntu', 'Mint']
Reloaded network: {'Cash': ('Cash', ['Ubuntu']), 'Ubuntu': ('Ubuntu', ['Cash', 'Mint']), 'Mint': ('Mint', ['Ubuntu'])}
Success: add user;
Success: duplicate user;
Success: add friendship;
Success: mutual friendship;
Success: distance two;
Success: cycle handling;
Success: reject self link;
Success: unknown user;
Success: missing friends;
Success: CSV round trip;
[STATUS] All extra-credit tests passed: True
PS E:\Misc School Projects\COP 4845-842 Python Programming\Homework2>

```


Problem 4: Movie Rankings

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Rank movie collaborations and actor box-office totals

import csv
import os

student_name = "Cesar Cabrera Garcia"
data_directory = os.path.dirname(os.path.abspath(__file__))
rated_filename = os.path.join(data_directory, "imdb-top-rated.csv")
grossing_filename = os.path.join(data_directory, "imdb-top-grossing.csv")
casts_filename = os.path.join(data_directory, "imdb-top-casts.csv")
display_limit = 10

def read_movies(filename: str) -> dict:
    """Index a ranked movie file by (title, year), retaining the value column."""
    movies = {}
    with open(filename, "r", newline="", encoding="utf-8-sig") as movie_file:
        reader = csv.reader(movie_file)
        next(reader, None)
        for row in reader:
            movies[(row[1], row[2])] = row[3]
    return movies

def read_casts(filename: str) -> dict:
    """Index the headerless cast file by (title, year)."""
    casts = {}
    with open(filename, "r", newline="", encoding="utf-8-sig") as cast_file:
        for row in csv.reader(cast_file):
            casts[(row[0], row[1])] = (row[2], row[3:])
    return casts

def ranked_totals(totals: dict) -> list:
    """Return (total, key) entries with the largest totals first."""
    ranking = []
    for key in totals:
        ranking.append((totals[key], key))
    ranking.sort(reverse=True)
    return ranking

def display_top_collaborations(limit: int = 0) -> None:
    """Display director/actor counts for rated movies; zero means all entries."""
    movies = read_movies(rated_filename)
    casts = read_casts(casts_filename)
    totals = {}
    shown = 0

    # Include the year in the key so remakes are not counted as the same movie.
    for movie in movies:
        if movie in casts:
            director, actors = casts[movie]
            counted_actors = []
            for actor in actors:
                if actor != "" and actor not in counted_actors:
                    key = (director, actor)
                    totals[key] = totals.get(key, 0) + 1
                    counted_actors.append(actor)
    print("Director / actor / top-rated movies")
    for total, key in ranked_totals(totals):
        if limit > 0 and shown >= limit:
            break
        shown = shown + 1
        print(shown, (key[0], key[1], total))
```

```

def display_top_actors(limit: int = 0) -> None:
    """Display actor box-office totals; zero means all entries."""
    movies = read_movies(grossing_filename)
    casts = read_casts(casts_filename)
    totals = {}
    shown = 0

    for movie in movies:
        if movie in casts:
            counted_actors = []
            for actor in casts[movie][1]:
                if actor != "" and actor not in counted_actors:
                    totals[actor] = totals.get(actor, 0) + int(movies[movie])
                    counted_actors.append(actor)
    print("Actor / total USA box office")
    for total, actor in ranked_totals(totals):
        if limit > 0 and shown >= limit:
            break
        shown = shown + 1
        print("{}: {}".format(shown, actor, total))

def main() -> None:
    """Display ten entries from each ranking using the provided CSV files."""
    print(student_name)
    print("[STATUS] Calculating movie rankings...")
    display_top_collaborations(display_limit)
    print()
    display_top_actors(display_limit)

if __name__ == "__main__":
    main()

```

Captured Program Output

```

Cesar Cabrera Garcia
[STATUS] Calculating movie rankings...
Director / actor / top-rated movies
1 ('Christopher Nolan', 'Christian Bale', 4)
2 ('Charles Chaplin', 'Charles Chaplin', 4)
3 ('Sergio Leone', 'Clint Eastwood', 3)
4 ('Peter Jackson', 'Sean Astin', 3)
5 ('Martin Scorsese', 'Robert De Niro', 3)
6 ('Martin Scorsese', 'Joe Pesci', 3)
7 ('Ingmar Bergman', 'Gunnar Björnstrand', 3)
8 ('Ingmar Bergman', 'Bibi Andersson', 3)
9 ('Francis Ford Coppola', 'Robert Duvall', 3)
10 ('Clint Eastwood', 'Clint Eastwood', 3)

Actor / total USA box office
1. Harrison Ford: $2,508,514,316
2. Daniel Radcliffe: $2,388,775,529
3. Tom Hanks: $2,129,067,581
4. Robert Downey Jr.: $2,059,220,121
5. Will Smith: $1,910,967,021
6. Johnny Depp: $1,819,530,977
7. Mike Myers: $1,681,680,289
8. Eddie Murphy: $1,651,627,240
9. Ben Stiller: $1,639,469,832
10. Tom Cruise: $1,504,567,044

```

```

PS E:\Misc School Projects\COP 4045-042 Python Programming\Homework2> & C:/Users/cedae/AppData/Local/Python/pythoncore-3.14-64/python.exe "e:/Misc School Projects/COP 4045-042 Python Programming/Homework2/p4_CabreraGarcia_Cesar.py"
Cesar Cabrera Garcia
[STATUS] Calculating movie rankings...
Director / actor / top-rated movies
1 ('Christopher Nolan', 'Christian Bale', 4)
2 ('Charles Chaplin', 'Charles Chaplin', 4)
3 ('Sergio Leone', 'Clint Eastwood', 3)
4 ('Peter Jackson', 'Sean Astin', 3)
5 ('Martin Scorsese', 'Robert De Niro', 3)
6 ('Martin Scorsese', 'Joe Pesci', 3)
7 ('Ingmar Bergman', 'Gunnar Björnstrand', 3)
8 ('Ingmar Bergman', 'Bibi Andersson', 3)
9 ('Francis Ford Coppola', 'Robert Duvall', 3)
10 ('Clint Eastwood', 'Clint Eastwood', 3)

Actor / total USA box office
1. Harrison Ford: $2,588,514,316
2. Daniel Radcliffe: $2,388,775,529
3. Tom Hanks: $2,129,067,581
4. Robert Downey Jr.: $2,059,228,121
5. Will Smith: $1,910,967,021
6. Johnny Depp: $1,819,530,977
7. Mike Myers: $1,681,680,289
8. Eddie Murphy: $1,651,627,240
9. Ben Stiller: $1,639,469,832
10. Tom Cruise: $1,584,567,044
PS E:\Misc School Projects\COP 4045-042 Python Programming\Homework2>

```

Problem 5: Weather Station Analyzer

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Validate weather observations and report station statistics

"""Analyze headerless station,date,temperature CSV files using local timestamps."""

import csv
from datetime import datetime
from pathlib import Path
from statistics import fmean
import sys

DATE_FORMAT = "%I:%M:%S %p %m/%d/%Y"
MIN_TEMPERATURE = -100.0
MAX_TEMPERATURE = 150.0
AUTHOR = "Cesar Cabrera Garcia"

Observations = dict[str, list[tuple[str, float]]]
Statistics = dict[str, tuple[float, float, float]]

def read_observations(filename: str) -> tuple[Observations, list[tuple[int, str]]]:
    """Read valid observations and collect errors using one-based line numbers.

    Dates retain their input spelling but are validated and sorted as datetimes.
    The first valid observation wins when a station/timestamp is repeated.
    File-access and encoding failures propagate to the caller.
    """
    observations: Observations = {}
    errors: list[tuple[int, str]] = []
    seen: set[tuple[str, datetime]] = set()

    with open(filename, encoding="utf-8-sig", newline="") as source:
        for line_number, line in enumerate(source, start=1):
            try:
                # Records occupy one physical line, allowing recovery after bad CSV.
                row = next(csv.reader([line], strict=True))
                if len(row) != 3:
                    raise ValueError("Expected station,date,temperature.")
                station, date_text, temperature_text = (value.strip() for value in row)
                if not station:
                    raise ValueError("Station name cannot be empty.")
                try:
                    timestamp = datetime.strptime(date_text, DATE_FORMAT)
                except ValueError as error:
                    raise ValueError("Invalid date; expected HH:MM:SS AM/PM MM/DD/YYYY.") from error
                try:
                    temperature = float(temperature_text)
                except ValueError as error:
                    raise ValueError("Temperature must be a number.") from error
                # This also rejects NaN and infinities without special cases.
                if not MIN_TEMPERATURE <= temperature <= MAX_TEMPERATURE:
                    raise ValueError("Temperature must be between -100.0 and 150.0.")
                key = (station, timestamp)
                if key in seen:
                    raise ValueError("Duplicate station/date observation.")
                seen.add(key)
                observations.setdefault(station, []).append((date_text, temperature))
            except (ValueError, csv.Error) as error:
                errors.append((line_number, str(error)))

    for records in observations.values():
        records.sort(key=lambda record: datetime.strptime(record[0], DATE_FORMAT))
    return observations, errors

def station_statistics(observations: Observations) -> Statistics:
    """Map each nonempty station to (minimum, maximum, mean) temperatures."""
```

```

statistics: Statistics = {}
for station, records in observations.items():
    if records:
        temperatures = [temperature for _, temperature in records]
        statistics[station] = (min(temperatures), max(temperatures), fmean(temperatures))
return statistics

def station_outliers(observations: Observations) -> dict[str, tuple[str, float, float]]:
    """Return stations whose chronologically latest temperature exceeds their mean."""
    statistics = station_statistics(observations)
    latest = {
        station: max(records, key=lambda record: datetime.strptime(record[0], DATE_FORMAT))
        for station, records in observations.items() if records
    }
    return {
        station: (latest[station][0], latest[station][1], statistics[station][2])
        for station in sorted(latest)
        if latest[station][1] > statistics[station][2]
    }

def write_statistics(filename: str, statistics: Statistics) -> None:
    """Write station,min,max,mean CSV rows alphabetically, with one decimal place.

    The output includes a header. Individual observations are not part of the
    statistics argument; input observations are ordered by read_observations.
    File-access failures propagate to the caller.
    """
    with open(filename, "w", encoding="utf-8", newline="") as destination:
        writer = csv.writer(destination)
        writer.writerow(("station", "min", "max", "mean"))
        for station in sorted(statistics):
            writer.writerow((station, *(f"{value:.1f}" for value in statistics[station])))

def main() -> int:
    """Read sys.argv paths, print results, and return a command-line exit status."""
    if len(sys.argv) != 3:
        print(f"Usage: {Path(sys.argv[0]).name} INPUT.csv OUTPUT.csv", file=sys.stderr)
        return 2

    source, destination = map(Path, sys.argv[1:])
    try:
        if source.resolve() == destination.resolve() or (
            destination.exists() and source.samefile(destination)
        ):
            print("[ERROR] Input and output must be different files.", file=sys.stderr)
            return 2
        observations, errors = read_observations(str(source))
        statistics = station_statistics(observations)
        outliers = station_outliers(observations)
        print(AUTHOR)
        for line_number, message in errors:
            print(f"[WARNING] Line {line_number}: {message}")
        print("Station | Minimum | Maximum | Mean")
        for station in sorted(statistics):
            minimum, maximum, mean = statistics[station]
            print(f"{station} | {minimum:.1f} | {maximum:.1f} | {mean:.1f}")
        print("Outliers (latest temperature above station mean):")
        for station, (date_text, temperature, mean) in outliers.items():
            print(f"{station} | {date_text} | {temperature:.1f} | mean {mean:.1f}")
        if not outliers:
            print("None")
        write_statistics(str(destination), statistics)
        print(f"[INFO] Wrote {len(statistics)} stations to {destination}.")
        return 0
    except (OSError, UnicodeError, csv.Error) as error:
        print(f"[ERROR] Could not process the weather files: {error}", file=sys.stderr)
        return 1

```

```
if __name__ == "__main__":
    sys.exit(main())
```

Problem 5: Unit Tests

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Test weather validation, statistics, ordering, and CLI errors
```

```
import contextlib
import csv
import io
from pathlib import Path
import tempfile
import unittest
from unittest.mock import patch
```

```
import p5_CabreraGarcia_Cesar as weather
```

```
class WeatherStationTests(unittest.TestCase):
    """Exercise file contracts and calculations using isolated temporary files."""

    def setUp(self):
        self.directory = tempfile.TemporaryDirectory()
        self.addCleanup(self.directory.cleanup)
        self.source = Path(self.directory.name) / "Cash.csv"
        self.destination = Path(self.directory.name) / "Mint.csv"

    def read_rows(self, text):
        self.source.write_text(text, encoding="utf-8")
        return weather.read_observations(str(self.source))

    def test_multiple_stations_and_chronological_order(self):
        observations, errors = self.read_rows(
            "Ubuntu,01:00:00 AM 01/01/2026,10\n"
            "Mint,12:00:00 PM 04/20/2026,-5\n"
            "Ubuntu,11:00:00 PM 12/31/2025,-10\n"
        )
        self.assertEqual(errors, [])
        self.assertEqual(list(observations), ["Ubuntu", "Mint"])
        self.assertEqual(observations["Ubuntu"][0], ("11:00:00 PM 12/31/2025", -10.0))

    def test_duplicates_use_parsed_dates_and_keep_first_valid(self):
        observations, errors = self.read_rows(
            "Cash,09:00:00 AM 04/20/2026,invalid\n"
            "Cash,09:00:00 AM 04/20/2026,1\n"
            "Cash,9:00:00 AM 4/20/2026,2\n"
            "Mint,09:00:00 AM 04/20/2026,3\n"
        )
        self.assertEqual([number for number, _ in errors], [1, 3])
        self.assertEqual(observations["Cash"], [("09:00:00 AM 04/20/2026", 1.0)])
        self.assertIn("Mint", observations)

    def test_temperature_boundaries_and_nonfinite_values(self):
        rows = "".join(
            f"Cash,09:00:00 AM 04/{day:02d}/2026,{value}\n"
            for day, value in enumerate(["-100", "150", "-100.1", "150.1", "nan", "inf", "-inf"], 1)
        )
        observations, errors = self.read_rows(rows)
        self.assertEqual([value for _, value in observations["Cash"]], [-100.0, 150.0])
        self.assertEqual([number for number, _ in errors], [3, 4, 5, 6, 7])

    def test_malformed_rows_invalid_dates_and_recovery(self):
        observations, errors = self.read_rows(
            "\n"
            "Cash,09:00:00 AM 04/20/2026\n"
            "Cash,09:00:00 AM 04/20/2026,1,2\n"
            "Cash,09:00:00 AM 02/30/2026,1\n"
            ",09:00:00 AM 04/20/2026,1\n"
        )
```

```

        "Cash,09:00:00 AM 04/20/2026,1\n"
        "Mint,09:00:00 AM 04/20/2026,7\n"
    )
    self.assertEqual([number for number, _ in errors], [1, 2, 3, 4, 5, 6])
    self.assertEqual(observations, {"Mint": [("09:00:00 AM 04/20/2026", 7.0)]})

def test_statistics_negative_temperatures_and_outliers(self):
    observations, _ = self.read_rows(
        "Cash,09:00:00 AM 04/20/2026,-10\n"
        "Cash,09:00:00 AM 04/21/2026,2\n"
        "Mint,09:00:00 AM 04/20/2026,3\n"
    )
    self.assertEqual(weather.station_statistics(observations), {
        "Cash": (-10.0, 2.0, -4.0), "Mint": (3.0, 3.0, 3.0),
    })
    # Latest must be determined by date even if a caller supplies unsorted data.
    observations["Cash"].reverse()
    self.assertEqual(weather.station_outliers(observations), {
        "Cash": ("09:00:00 AM 04/21/2026", 2.0, -4.0),
    })

def test_sorted_output_and_exact_decimal_precision(self):
    weather.write_statistics(str(self.destination), {
        "Zorin OS": (0.0, 1.0, 1 / 3), "Cash": (-12.25, 10, -1.125),
    })
    with self.destination.open(newline="", encoding="utf-8") as output:
        self.assertEqual(list(csv.reader(output)), [
            ["station", "min", "max", "mean"],
            ["Cash", "-12.2", "10.0", "-1.1"],
            ["Zorin OS", "0.0", "1.0", "0.3"],
        ])

def test_missing_input_and_unwritable_output(self):
    with self.assertRaises(FileNotFoundError):
        weather.read_observations(str(self.source))
    with self.assertRaises(PermissionError):
        weather.write_statistics(self.directory.name, {})

def test_empty_file_and_empty_station(self):
    self.assertEqual(self.read_rows(""), ({}, []))
    self.assertEqual(weather.station_statistics({"Cash": []}), {})
    self.assertEqual(weather.station_outliers({"Cash": []}), {})
    weather.write_statistics(str(self.destination), {})
    self.assertEqual(self.destination.read_text(), "station,min,max,mean\n")

def test_cli_file_error_is_graceful(self):
    with patch("sys.argv", ["weather", str(self.source), str(self.destination)]):
        with contextlib.redirect_stderr(io.StringIO()) as output:
            self.assertEqual(weather.main(), 1)
    self.assertIn("ERROR", output.getvalue())

def test_cli_success_and_source_overwrite_protection(self):
    text = "Cash,09:00:00 AM 04/20/2026,2\n"
    self.source.write_text(text, encoding="utf-8")
    with patch("sys.argv", ["weather", str(self.source), str(self.destination)]):
        with contextlib.redirect_stdout(io.StringIO()):
            self.assertEqual(weather.main(), 0)
    self.assertTrue(self.destination.exists())
    with patch("sys.argv", ["weather", str(self.source), str(self.source)]):
        with contextlib.redirect_stderr(io.StringIO()):
            self.assertEqual(weather.main(), 2)
    self.assertEqual(self.source.read_text(), text)

if __name__ == "__main__":
    unittest.main()

```

Captured Program Output

```
Cesar Cabrera Garcia
Station | Minimum | Maximum | Mean
Cash | -10.0 | 2.0 | -4.0
Mint | 50.0 | 50.0 | 50.0
Ubuntu | 68.0 | 72.0 | 70.0
Zorin OS | 58.0 | 60.0 | 59.0
Outliers (latest temperature above station mean):
Cash | 09:28:09 AM 04/21/2026 | 2.0 | mean -4.0
Ubuntu | 09:28:09 AM 04/21/2026 | 72.0 | mean 70.0
[INFO] Wrote 4 stations to E:\Misc School Projects\COP 4045-042 Python Programming\Homework2\p5\statistics.csv.

test_cli_file_error_is_graceful (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_cli_file_error_is_graceful)
... ok
test_cli_success_and_source_overwrite_protection (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_cli_success
_and_source_overwrite_protection) ... ok
test_duplicates_use_parsed_dates_and_keep_first_valid (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_duplic
ates_use_parsed_dates_and_keep_first_valid) ... ok
test_empty_file_and_empty_station (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_empty_file_and_empty_stati
on) ... ok
test_malformed_rows_invalid_dates_and_recovery (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_malformed_row
s_invalid_dates_and_recovery) ... ok
test_missing_input_and_unwritable_output (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_missing_input_and_u
nwritable_output) ... ok
test_multiple_stations_and_chronological_order (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_multiple_stat
ions_and_chronological_order) ... ok
test_sorted_output_and_exact_decimal_precision (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_sorted_output
_and_exact_decimal_precision) ... ok
test_statistics_negative_temperatures_and_outliers (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_statistic
s_negative_temperatures_and_outliers) ... ok
test_temperature_boundaries_and_nonfinite_values (p5_CabreraGarcia_Cesar_test.WeatherStationTests.test_temperature
_boundaries_and_nonfinite_values) ... ok

-----
Ran 10 tests in 0.010s

OK
```


Problem 5: Coding-Agent Prompt

help with homework 2 problem 5
i need a python program called p5_CabreraGarcia_Cesar.py for a weather station analyzer
it needs to read station,date,temperature from a file
use datetime for the dates
temps only valid from -100 to 150
bad lines and duplicate station/date entries should go into an errors list
observations need to be grouped by station and sorted by date
make station_statistics for min max and average
make station_outliers use station_statistics and a dictionary comprehension. latest temp higher than mean counts as an outlier
make write_statistics sort stations alphabetically and write numbers with 1 decimal place
add at least 6 unittest tests for multiple stations, negative temps, duplicates, invalid temps, statistics, sorted output and missing files
main should use sys.argv for input and output filenames, print stats and outliers, write the output file, and handle file errors
keep it simple and readable
put my name Cesar Cabrera Garcia in the file and in your response

Problem 5: Coding-Agent Response

Cesar Cabrera Garcia, your existing program meets these requirements. Your name is included in the file and printed when it runs.

- Python program
- Unit tests

All 10 tests pass, covering validation, duplicates, chronological sorting, statistics, outliers, output formatting, and file errors.

Run from the project folder:

</> powershell

python Homework2/p5/p5_CabreraGarcia_Cesar.py Homework2/p5/observations.csv Homework2/p5/statistics.csv

Input dates use 09:28:09 AM 04/20/2026 . Invalid records are collected as (line_number, error_message) entries while valid records continue processing.